LAPORAN PRAKTIKUM

PRAKTIKUM PENAMBANGAN DATA

PERTEMUAN 10

CONVOLUTIONAL NEURAL NETWORK (CNN)



Disusun oleh:

Nama	:	Hanan Fijananto
NIM	:	24/538946/SV/24555
Kelas	:	A2
Dosen Pengampu	:	Dr.Imam Fahrurrozi, S.T., M.Cs.

PROGRAM STUDI D-IV TEKNOLOGI REKAYASA PERANGKAT LUNAK

DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA

SEKOLAH VOKASI

UNIVERSITAS GADJAH MADA

YOGYAKARTA

2026

A Tujuan

1. Memahami keterbatasan Multi-Layer Perceptron (MLP) dalam pengelolaan citra dan keunggulan CNN.

2. Menjelaskan arsitektur dasar CNN: Convolution Layer, Pooling Layer, Flattening, dan Fully Connected Layer.

3. Membangun model CNN sederhana menggunakan framework Keras/TensorFlow.

4. Menerapkan teknik regularisasi Dropout untuk mencegah overfitting.

B Dasar Teori

B.1 Representasi Citra dan Keterbatasan MLP

Citra digital direpresentasikan sebagai matriks nilai intensitas piksel multidimensi, mulai dari citra skala abu-abu berdimensi dua hingga citra berwarna berdimensi tiga dengan kanal warna (misalnya RGB) [1]. Pada arsitektur *Multi-Layer Perceptron* (MLP), citra masukan harus diratakan (*flattening*) menjadi vektor satu dimensi sebelum dialirkan ke lapisan terhubung penuh (*fully connected layer*). Proses ini menghilangkan hubungan spasial dan topologi lokal antarpiksel tetangga, padahal informasi geometris seperti tepi, tekstur, dan kontur sangat krusial dalam pengenalan objek visual [2].

Selain hilangnya keterkaitan spasial, keterbatasan mendasar MLP terletak pada membengkaknya jumlah parameter bobot secara drastis seiring bertambahnya resolusi citra, yang memicu masalah *curse of dimensionality* dan risiko tinggi terjadinya *overfitting* [1]. Struktur MLP juga tidak memiliki sifat *translation invariance*; pergeseran posisi objek sekecil apa pun pada citra dapat menghasilkan representasi vektor

yang sama sekali berbeda sehingga model kesulitan mengenali objek secara konsisten.

B.2 Arsitektur Convolutional Neural Network (CNN)

Convolutional Neural Network (CNN) merupakan arsitektur jaringan saraf mendalam yang secara khusus dirancang untuk memproses data berstruktur kisi spasial dua atau tiga dimensi [1, 3]. Arsitektur CNN umumnya terbagi menjadi tahap ekstraksi fitur dan tahap klasifikasi. Pada tahap ekstraksi fitur, *convolutional layer* menerapkan filter atau kernel geser dengan konsep *parameter sharing* dan *local receptive fields* untuk menangkap pola lokal secara otomatis, sedangkan *pooling layer* (seperti *max pooling*) mereduksi resolusi spasial peta fitur (*feature map*) guna menekan beban komputasi serta memberikan toleransi terhadap distorsi dan translasi kecil [2].

Setelah representasi fitur abstrak terbentuk, peta fitur diubah menjadi vektor satu dimensi melalui operasi *flattening* dan diteruskan ke *fully connected layer* sebagai blok klasifikasi [3]. Lapisan terhubung penuh ini memetakan kombinasi fitur-fitur kompleks ke ruang probabilitas kelas target menggunakan fungsi aktivasi keluaran seperti *Softmax* untuk klasifikasi multikelas.

B.3 Drop Out

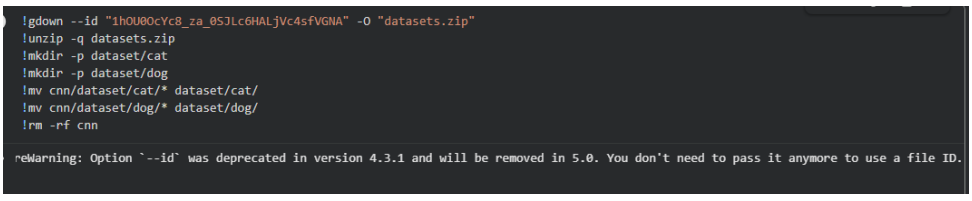
Dropout adalah teknik regularisasi komputasi yang dirancang untuk mencegah terjadinya fenomena *overfitting* pada jaringan saraf tiruan berdimensi besar [4]. Selama siklus pelatihan (*training*), *dropout* bekerja dengan cara mematikan (*dropping out*) unit-unit neuron beserta koneksi keluar-masuknya secara acak berdasarkan probabilitas retensi tertentu (p). Mekanisme penonaktifan acak ini memaksa jaringan untuk tidak mengandalkan interaksi antar-neuron secara kaku (*co-adaptation*), sehingga setiap neuron terdorong mempelajari representasi fitur yang lebih mandiri dan kuat [4].

Secara matematis dan konseptual, penerapan *dropout* dapat dipandang sebagai bentuk aproksimasi efisien dari teknik *ensemble learning* atas ribuan kombinasi sub-arsitektur jaringan saraf yang berbeda tanpa memerlukan biaya komputasi model jamak [4, 3]. Pada tahap pengujian atau inferensi (*testing/inference*), seluruh neuron diaktifkan kembali tanpa ada yang dinonaktifkan, di mana bobot keluaran diskalakan secara proporsional sesuai probabilitas retensi agar menghasilkan estimasi prediksi rata-rata ansambel yang konsisten.

C Hasil dan Pembahasan

C.1 Pengunduhan dan Penataan Direktori Dataset

Tahap awal praktikum diawali dengan mempersiapkan dataset citra hewan yang terdiri atas dua kelas target, yaitu kucing (*cat*) dan anjing (*dog*). Berkas arsip dataset diunduh dari Google Drive menggunakan perintah `gdown`, kemudian diekstraksi ke dalam lingkungan kerja. Selanjutnya, dibuat struktur direktori baru `dataset/cat` dan `dataset/dog`, lalu citra dari masing-masing kategori dipindahkan ke direktori yang sesuai sebelum direktori sementara dibersihkan, seperti yang ditunjukkan pada Gambar 1. Penataan ini diperlukan agar pemuatan data berbasis subdirektori dapat dilakukan secara otomatis oleh Keras.

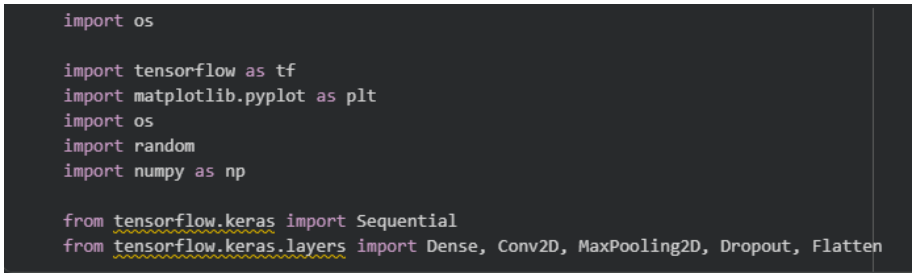


Gambar 1: Proses Pengunduhan dan Penataan Direktori Dataset

C.2 Import Pustaka dan Modul Dependensi

Langkah berikutnya adalah mengimpor pustaka dan dependensi yang diperlukan ke dalam lingkungan kerja Python (Gambar 2). Pustaka standar `os` digunakan

untuk manajemen berkas dan jalur direktori, `tensorflow` sebagai kerangka kerja utama pembelajaran mendalam, `matplotlib.pyplot` untuk kebutuhan visualisasi data dan grafik metrik, serta `random` dan `numpy` untuk manipulasi larik numerik. Selain itu, diimpor modul `Sequential` dari Keras beserta komponen lapisan konvolusi seperti `Conv2D`, `MaxPooling2D`, `Dropout`, `Flatten`, dan `Dense`.



```
import os

import tensorflow as tf
import matplotlib.pyplot as plt
import os
import random
import numpy as np

from tensorflow.keras import Sequential
from tensorflow.keras.layers import Dense, Conv2D, MaxPooling2D, Dropout, Flatten
```

Gambar 2: Pengimporan Pustaka dan Modul Pemodelan CNN

C.3 Prapemrosesan dan Pembagian Dataset (Splitting)

Pada tahap prapemrosesan, dimensi citra masukan ditetapkan seragam sebesar 224×224 piksel (`IMAGE_SIZE = 224`) dengan ukuran batch pemrosesan sebesar 64. Modul `ImageDataGenerator` diinisialisasi dengan penskalaan ulang (*rescaling*) nilai piksel sebesar $1/255$ agar intensitas piksel berada pada rentang $[0, 1]$, serta pemisahan data validasi sebesar 20% melalui argumen `validation_split=0.2`. Berdasarkan Gambar 3, generator berhasil mendeteksi dan membagi 100 citra menjadi 80 citra untuk data latih (*training*) dan 20 citra untuk data validasi (*validation*) dari 2 kelas.

```
data_path = "dataset/"
base_dir = os.path. join(data_path)

IMAGE_SIZE = 224 #model input size
BATCH_SIZE = 64

datagen = tf.keras.preprocessing. image. ImageDataGenerator(
rescale = 1./255,
validation_split=0.2)

train_generator = datagen. flow_from_directory(
base_dir,
target_size=(IMAGE_SIZE, IMAGE_SIZE),
batch_size = BATCH_SIZE,
subset='training')

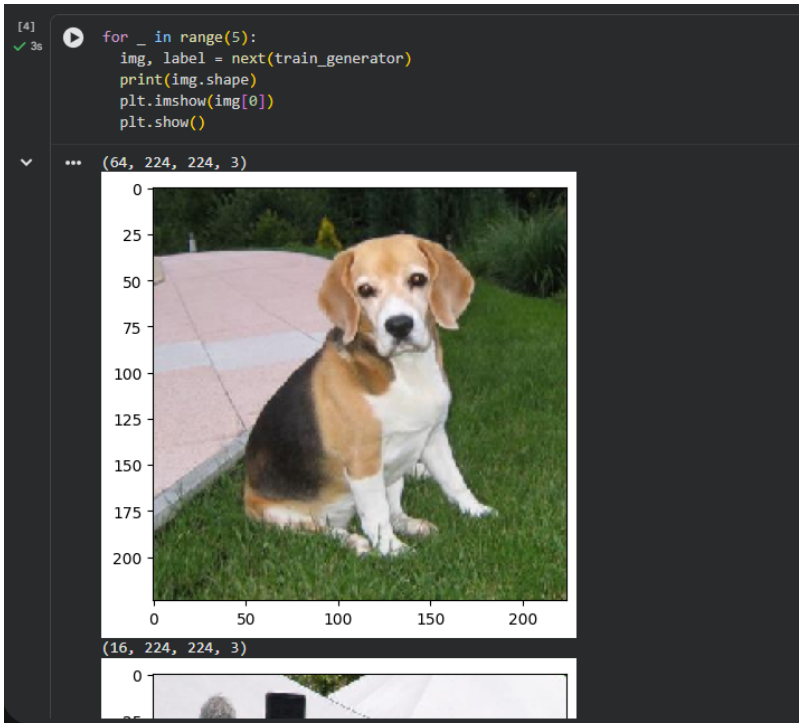
val_generator = datagen.flow_from_directory(
base_dir,
target_size=(IMAGE_SIZE, IMAGE_SIZE),
batch_size = BATCH_SIZE,
subset='validation')
```

```
Found 80 images belonging to 2 classes.
Found 20 images belonging to 2 classes.
```

Gambar 3: Konfigurasi *ImageDataGenerator* dan Pembagian Data Latih serta Validasi

C.4 Pembacaan Batch dan Visualisasi Sampel Data

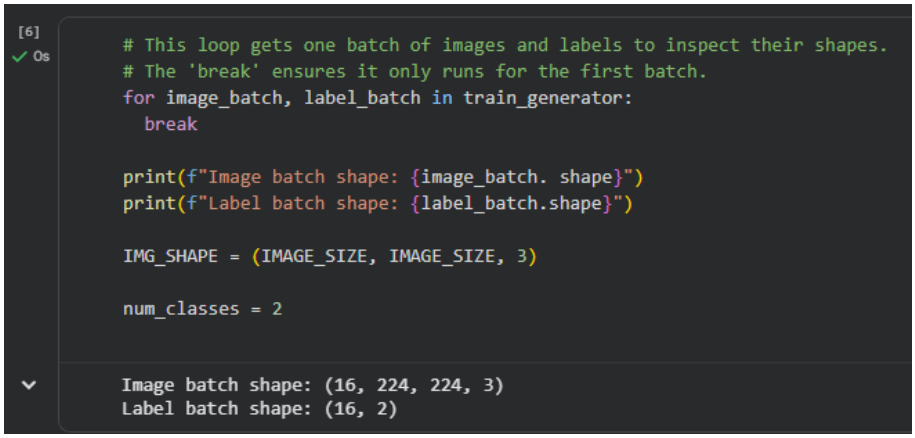
Untuk memastikan bahwa citra terbaca secara tepat oleh generator, dilakukan iterasi pemanggilan batch data latih menggunakan perintah `next(train_generator)` dan divisualisasikan dengan `plt.imshow()` sebagaimana tampak pada Gambar 4. Hasil eksekusi menampilkan citra anjing beresolusi $224 \times 224 \times 3$, dengan ukuran batch pertama memuat 64 citra berdimensi $(64, 224, 224, 3)$ dan batch kedua memuat sisa 16 citra dengan dimensi $(16, 224, 224, 3)$ sesuai total 80 data latih.



Gambar 4: Pembacaan dan Visualisasi Sampel Citra Data Latih

C.5 Validasi Bentuk Dimensi Batch dan Label

Sebelum arsitektur jaringan dibangun, dilakukan validasi terhadap bentuk dimensi (*shape*) dari tensor citra dan vektor label untuk memastikan kesiapan data dalam proses komputasi tensor. Seperti yang diperlihatkan pada Gambar 5, inspeksi batch menghasilkan dimensi matriks citra sebesar $(16, 224, 224, 3)$ dan label berformat *one-hot encoding* berdimensi $(16, 2)$. Selain itu, variabel bentuk masukan `IMG_SHAPE` didefinisikan sebesar $(224, 224, 3)$ dan jumlah kelas sasaran `num_classes` bernilai 2.



Gambar 5: Validasi Dimensi Tensor Citra dan Matriks Label Satu Batch

C.6 Pembangunan Arsitektur Model CNN

Arsitektur model CNN dirancang secara sekuensial menggunakan dua blok konvolusi utama (Gambar 6). Blok pertama memuat dua lapisan `Conv2D` dengan 32 filter berukuran 3×3 dan aktivasi ReLU, diikuti oleh `MaxPooling2D(2, 2)` untuk reduksi spasial serta lapisan regularisasi `Dropout(0.1)` untuk mencegah ketergantungan antar-neuron. Blok kedua menerapkan dua lapisan `Conv2D` dengan 64 filter berukuran 3×3 dan `MaxPooling2D(2, 2)`. Fitur konvolusi kemudian diratakan melalui `Flatten()`, dialirkan ke lapisan `Dense(512, activation='relu')`, dan berakhir pada lapisan keluaran `Dense(2, activation='softmax')`. Model dikompilasi dengan pengoptimal *RMSprop*, fungsi *loss categorical crossentropy*, dan metrik evaluasi akurasi.

```
model = Sequential()

#
# Conv Block 1: 32 Filters, MaxPool.

model.add(Conv2D(filters=32, kernel_size=3, padding='same', activation='relu', input_shape=IMG_SHAPE))
model.add(Conv2D(filters=32, kernel_size=3, padding='same', activation='relu'))
model.add(MaxPooling2D(pool_size=(2, 2)))
model.add(Dropout(0.1))

# Conv Block 2: 64 Filters, MaxPool.

model.add(Conv2D(filters=64, kernel_size=3, padding='same', activation='relu'))
model.add(Conv2D(filters=64, kernel_size=3, padding='same', activation='relu'))
model.add(MaxPooling2D(pool_size=(2, 2)))

# Flatten the convolutional features.
#
model.add(Flatten())
model.add(Dense(512, activation='relu'))
model.add(Dense(num_classes, activation='softmax'))

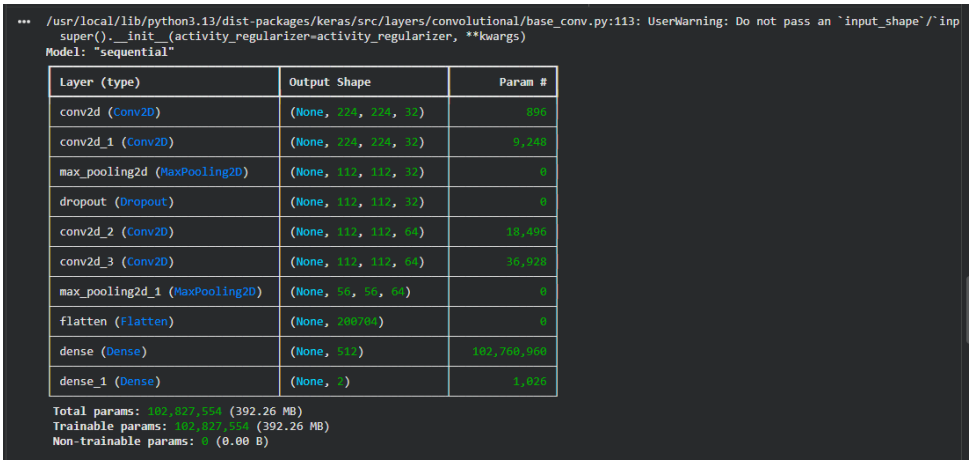
model.compile(optimizer='rmsprop',
loss='categorical_crossentropy',
metrics=['accuracy' ],)

model.summary()
```

Gambar 6: Implementasi Kode Pembangunan dan Kompilasi Arsitektur CNN

C.7 Ringkasan Parameter Model (*Model Summary*)

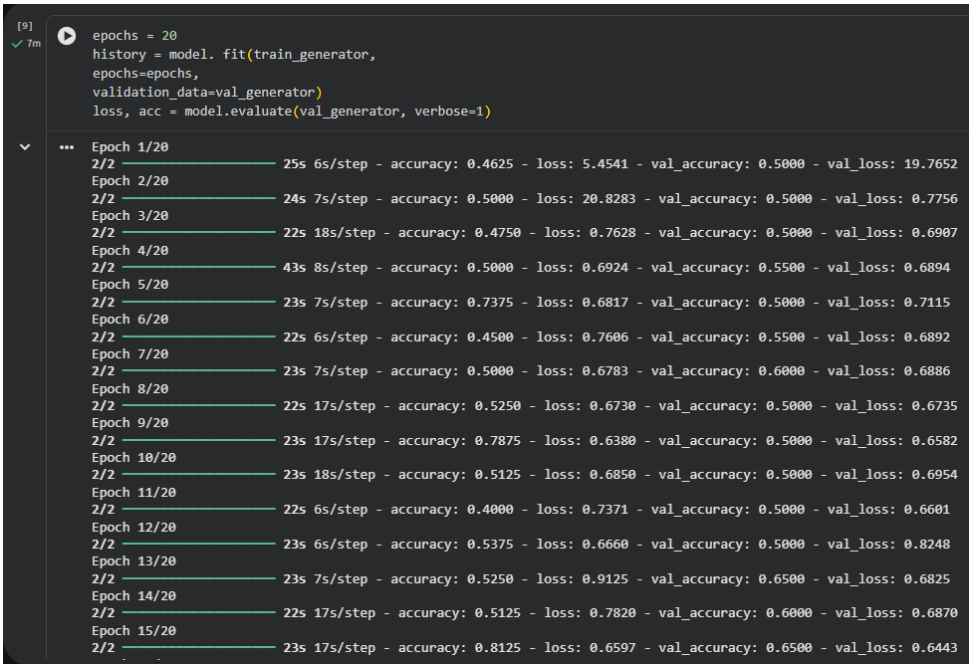
Pemeriksaan ringkasan arsitektur menggunakan perintah `model.summary()` ditampilkan pada Gambar 7. Lapisan konvolusi dan pooling mereduksi dimensi spasial secara berjenjang dari $(224, 224, 32)$ menjadi $(56, 56, 64)$ sebelum ditransformasikan oleh `Flatten` menjadi vektor satu dimensi dengan 200.704 fitur ($56 \times 56 \times 64$). Hubungan penuh antara lapisan *flatten* menuju lapisan `Dense(512)` menghasilkan parameter terbesar yaitu sebanyak 102.760.960 bobot. Total parameter yang dilatih pada model ini tercatat sebanyak 102.827.554 parameter (setara 392,26 MB).



Gambar 7: Ringkasan Struktur Lapisan dan Jumlah Parameter Model CNN

C.8 Proses Pelatihan Model (*Model Training*)

Proses pelatihan model CNN dijalankan sebanyak 20 epoch menggunakan perintah `model.fit()` dengan melibatkan generator data latih dan generator data validasi (Gambar 8). Karena batch size yang digunakan, setiap epoch memproses data latih dalam 2 langkah komputasi (*steps*). Pada epoch pertama, model mencatatkan nilai *loss* sebesar 5.4541 dan akurasi 0.4625. Seiring berjalannya pelatihan, performa model menunjukkan perbaikan yang nyata di mana akurasi pelatihan mencapai 0.8125 pada epoch ke-15 dengan nilai *loss* yang menurun secara konsisten.



Gambar 8: Progres Pelatihan Model CNN Selama 20 Epoch

C.9 Implementasi Kode Evaluasi dan Visualisasi Metrik

Setelah proses pelatihan selesai, data performa historis yang tercatat pada atribut `history.history` diekstraksi ke variabel `acc`, `val_acc`, `loss`, dan `val_loss`. Sebagaimana tampak pada Gambar 9, modul visualisasi `matplotlib.pyplot` digunakan untuk menyusun dua subplot bersusun berukuran 8×8: panel atas memetakan kurva akurasi data latih dan validasi, sedangkan panel bawah memetakan kurva *cross entropy loss* terhadap setiap epoch.

```
[10]
✓ 0s
acc = history.history['accuracy']
val_acc = history.history['val_accuracy' ]
loss = history.history['loss' ]
val_loss = history.history['val_loss' ]

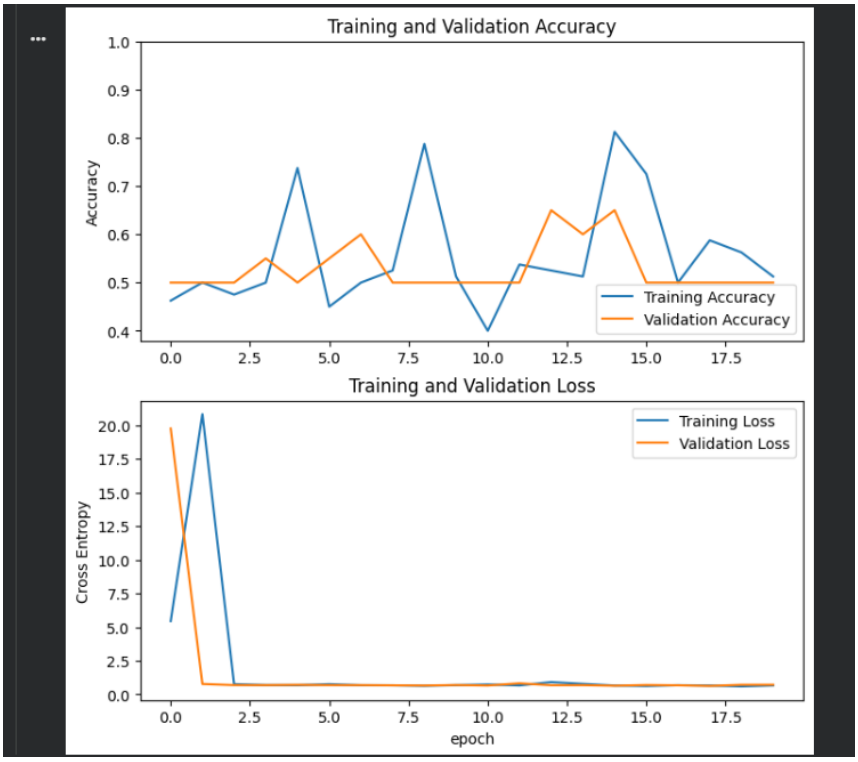
plt.figure(figsize=(8, 8))
plt.subplot(2, 1, 1)
plt.plot(acc, label='Training Accuracy')
plt.plot(val_acc, label='Validation Accuracy')
plt. legend(loc='lower right')
plt.ylabel('Accuracy')
plt.ylim([min(plt.ylim()),1])
plt. title('Training and Validation Accuracy')

plt.subplot(2, 1, 2)
plt.plot(loss, label='Training Loss')
plt.plot(val_loss, label='Validation Loss')
plt. legend(loc='upper right')
plt.ylabel('Cross Entropy')
plt.title('Training and Validation Loss')
plt.xlabel('epoch')
plt. show()
```

Gambar 9: Implementasi Kode Visualisasi Metrik Akurasi dan *Loss*

C.10 Analisis Grafik Visualisasi Hasil Evaluasi Model

Grafik evaluasi model CNN selama 20 epoch disajikan secara lengkap pada Gambar 10. Pada grafik akurasi (atas), akurasi data latih (garis biru) mengalami peningkatan berfluktuasi hingga menyentuh nilai puncak sekitar 81% sebelum mengalami koreksi pada epoch terakhir, sedangkan akurasi validasi (garis oranye) bergerak stabil dalam rentang 50% hingga 65%. Pada grafik kerugian (bawah), kurva *training loss* dan *validation loss* mengalami penurunan tajam dari nilai awal yang sangat tinggi ($\sim$ 20 pada validasi dan $\sim$ 5.4 pada pelatihan) menuju kisaran konvergen di bawah 1.0 sejak epoch kedua dan bertahan stabil hingga akhir pelatihan. Dinamika fluktuasi akurasi ini dipengaruhi oleh ukuran dataset yang terbatas (100 citra), namun penurunan nilai *loss* membuktikan bahwa filter konvolusi dan regularisasi *Dropout* berhasil mempelajari representasi visual pola citra dengan baik.



Gambar 10: Grafik Perbandingan Akurasi dan *Loss* antara Data Latih dan Validasi

D Tugas dan Analisis

D.1 Pemilihan dan Eksplorasi Dataset Citra

Sesuai dengan instruksi tugas pertama, dilakukan pemilihan dataset citra berwarna dengan ukuran ringkas untuk memfasilitasi proses eksperimen yang efisien. Dataset yang dipilih adalah subset dari **CIFAR-10** yang difokuskan pada permasalahan klasifikasi multikelas (*multi-class classification*) untuk kategori moda transportasi/kendaraan, yaitu:

1. **Kelas 0 (Pesawat Terbang / *Airplane*)**
2. **Kelas 1 (Mobil / *Automobile*)**
3. **Kelas 2 (Kapal Laut / *Ship*)**

Setiap sampel citra memiliki resolusi spasial 32×32 piksel dengan 3 kanal warna (RGB). Dari dataset mentah, dilakukan penyaringan (*filtering*) sehingga diperoleh total 15.000 sampel data latih (5.000 citra per kelas) dan 3.000 sampel data uji (1.000 citra per kelas). Prapemrosesan data dilakukan melalui normalisasi nilai intensitas piksel $[0, 255]$ ke dalam skala interval $[0.0, 1.0]$ dengan membagi nilai piksel terhadap 255.0 guna mempercepat konvergensi gradien selama proses optimasi. Visualisasi sampel citra untuk masing-masing kelas disajikan pada Gambar 12.

```
# Muat dataset mentah CIFAR-10
(X_train_raw, y_train_raw), (X_test_raw, y_test_raw) = cifar10.load_data()

# Pilih 3 kelas kendaraan: 0 = Airplane, 1 = Automobile, 8 = Ship
target_classes = [0, 1, 8]
class_names = ["Airplane", "Automobile", "Ship"]

def filter_classes(X, y, selected_classes):
    mask = np.isin(y.flatten(), selected_classes)
    X_filtered = X[mask]
    y_filtered = y[mask].flatten()
    # Remap label ke 0, 1, 2
    mapping = {selected_classes[i]: i for i in range(len(selected_classes))}
    y_remapped = np.array([mapping[val] for val in y_filtered])
    return X_filtered, y_remapped

X_train_sub, y_train_sub = filter_classes(X_train_raw, y_train_raw, target_classes)
X_test_sub, y_test_sub = filter_classes(X_test_raw, y_test_raw, target_classes)

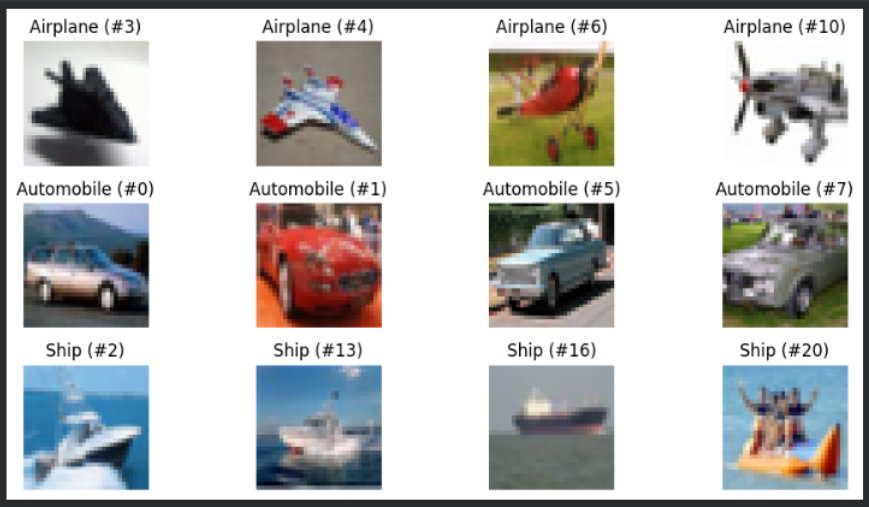
# Normalisasi nilai piksel [0, 255] ke rentang [0.0, 1.0]
X_train = X_train_sub.astype("float32") / 255.0
X_test = X_test_sub.astype("float32") / 255.0
y_train = y_train_sub
y_test = y_test_sub

print("=== INFORMASI DATASET ===")
print(f"Data Latih (X_train): {X_train.shape}, Label: {y_train.shape}")
print(f"Data Uji   (X_test) : {X_test.shape}, Label: {y_test.shape}")
print(f"Jumlah Kelas        : {len(class_names)} ({class_names})")
```

```
Downloading data from https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz
170498071/170498071 ━━━━━━━━━━━━━━━━━━━━ 108s 1us/step
=== INFORMASI DATASET ===
Data Latih (X_train): (15000, 32, 32, 3), Label: (15000,)
Data Uji   (X_test) : (3000, 32, 32, 3), Label: (3000,)
Jumlah Kelas        : 3 (['Airplane', 'Automobile', 'Ship'])
```

Gambar 11: Kode untuk Load dataset dan label kelas

```python
# Visualisasi sampel data untuk setiap kelas
plt.figure(figsize=(10, 5))
for idx, c_name in enumerate(class_names):
    sample_indices = np.where(y_train == idx)[0][:4]
    for col, sample_idx in enumerate(sample_indices):
        plt.subplot(3, 4, idx * 4 + col + 1)
        plt.imshow(X_train[sample_idx])
        plt.title(f"{c_name} (#{sample_idx})")
        plt.axis("off")
plt.tight_layout()
plt.show()
```



Gambar 12: Visualisasi Sampel Citra Dataset Kendaraan (*Airplane*, *Automobile*, dan *Ship*)

D.2 Perancangan dan Pelatihan Arsitektur Model CNN

Untuk menyelesaikan tugas kedua, dirancang arsitektur model *Convolutional Neural Network* (CNN) hierarkis yang mengintegrasikan teknik regularisasi modern untuk mencegah *overfitting*. Arsitektur model terdiri atas:

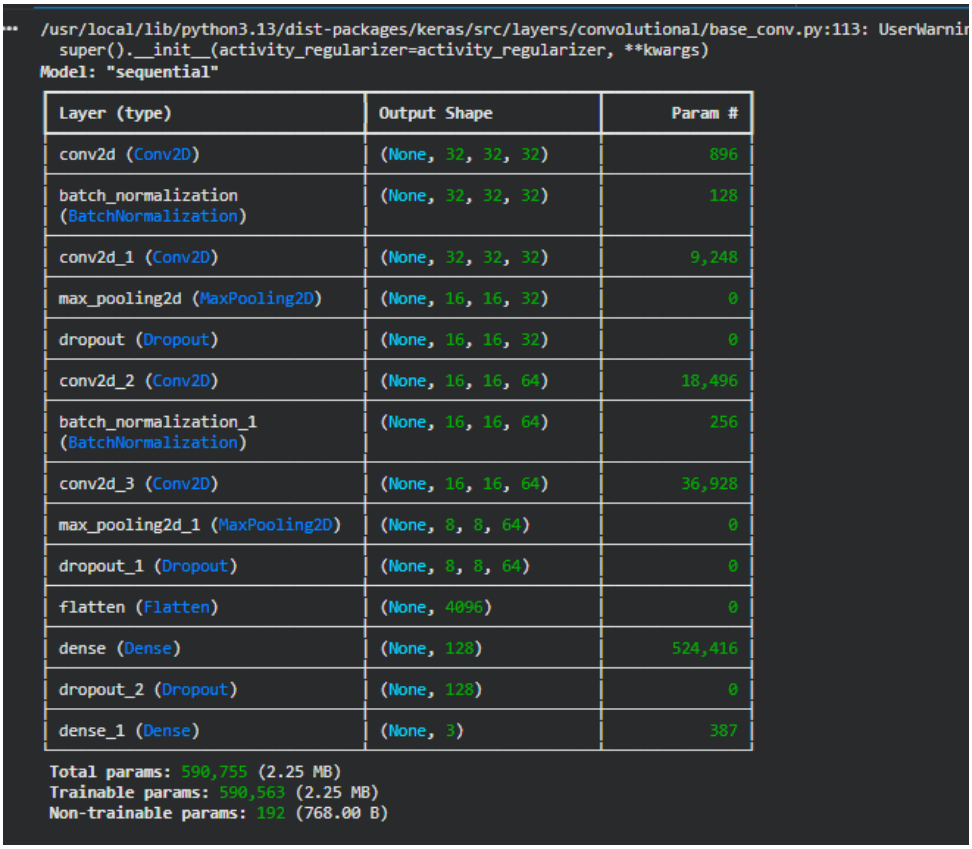
1. **Blok Konvolusi 1:** Lapisan `Conv2D` (32 filter, kernel 3×3, padding *'same'*, aktivasi ReLU) masukan $(32, 32, 3)$, lapisan `BatchNormalization()`, lapisan `Conv2D` (32 filter), lapisan `MaxPooling2D(2, 2)`, serta `Dropout(0.25)`.

2. **Blok Konvolusi 2:** Lapisan `Conv2D` (64 filter, kernel 3×3, padding *'same'*, aktivasi ReLU), lapisan `BatchNormalization()`, `Conv2D` (64 filter), `MaxPooling2D(2, 2)`, serta `Dropout(0.25)`.

3. **Blok Klasifikasi:** Lapisan `Flatten()` perata fitur, lapisan terhubung penuh

`Dense` (128 unit, ReLU), lapisan `Dropout(0.40)`, serta lapisan keluaran `Dense` (3 unit, fungsi aktivasi *Softmax*) untuk klasifikasi multikelas.

Model dikompilasi menggunakan pengoptimal *Adam* (*learning rate* = 0.001) dengan fungsi kerugian *sparse categorical crossentropy* dan metrik evaluasi akurasi. Ringkasan struktur arsitektur dan parameter model diperlihatkan pada Gambar 13. Model kemudian dilatih selama 20 epoch dengan ukuran batch 64 menggunakan data latih dan langsung divalidasi terhadap data uji sebagaimana tampak pada Gambar 14.

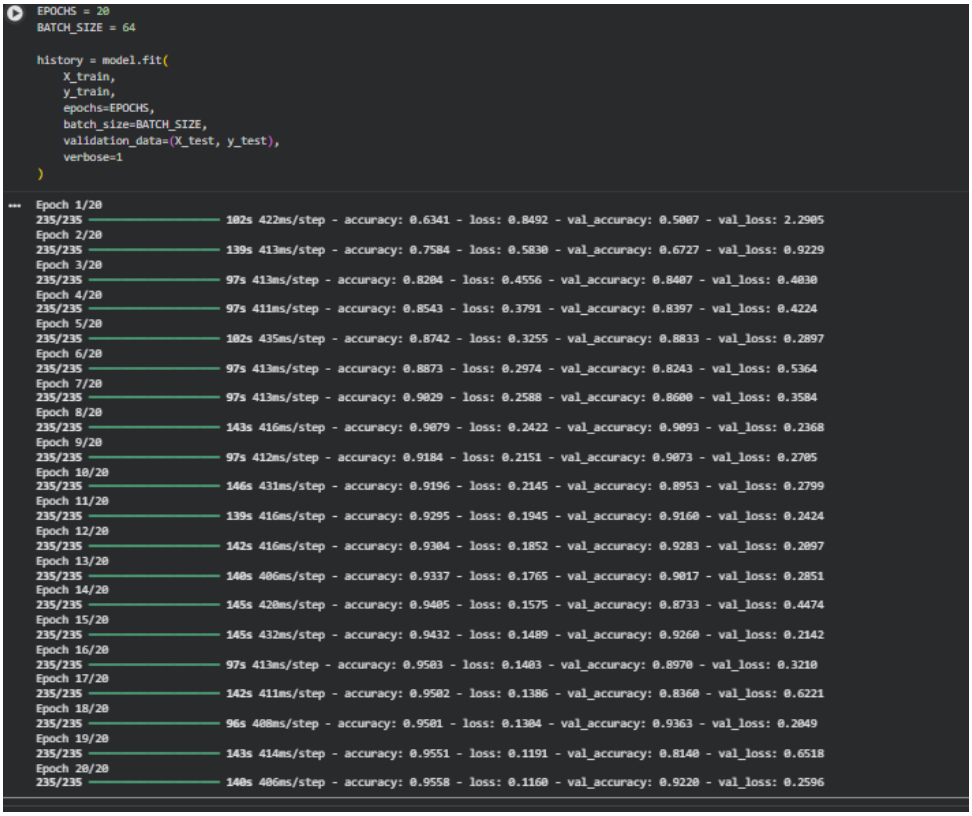


```
/usr/local/lib/python3.13/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarnir
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential"
```

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 32, 32, 32)	896
batch_normalization (BatchNormalization)	(None, 32, 32, 32)	128
conv2d_1 (Conv2D)	(None, 32, 32, 32)	9,248
max_pooling2d (MaxPooling2D)	(None, 16, 16, 32)	0
dropout (Dropout)	(None, 16, 16, 32)	0
conv2d_2 (Conv2D)	(None, 16, 16, 64)	18,496
batch_normalization_1 (BatchNormalization)	(None, 16, 16, 64)	256
conv2d_3 (Conv2D)	(None, 16, 16, 64)	36,928
max_pooling2d_1 (MaxPooling2D)	(None, 8, 8, 64)	0
dropout_1 (Dropout)	(None, 8, 8, 64)	0
flatten (Flatten)	(None, 4096)	0
dense (Dense)	(None, 128)	524,416
dropout_2 (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 3)	387

```
Total params: 590,755 (2.25 MB)
Trainable params: 590,563 (2.25 MB)
Non-trainable params: 192 (768.00 B)
```

Gambar 13: Ringkasan Lapisan dan Parameter Model CNN Klasifikasi Kendaraan



```
EPOCHS = 20
BATCH_SIZE = 64

history = model.fit(
    X_train,
    y_train,
    epochs=EPOCHS,
    batch_size=BATCH_SIZE,
    validation_data=(X_test, y_test),
    verbose=1
)
```

```
Epoch 1/20
235/235 ━━━━━━━━━━━━━━━━━━━━ 102s 422ms/step - accuracy: 0.6341 - loss: 0.8492 - val_accuracy: 0.5007 - val_loss: 2.2905
Epoch 2/20
235/235 ━━━━━━━━━━━━━━━━━━━━ 139s 413ms/step - accuracy: 0.7584 - loss: 0.5830 - val_accuracy: 0.6727 - val_loss: 0.9229
Epoch 3/20
235/235 ━━━━━━━━━━━━━━━━━━━━ 97s 413ms/step - accuracy: 0.8204 - loss: 0.4556 - val_accuracy: 0.8407 - val_loss: 0.4030
Epoch 4/20
235/235 ━━━━━━━━━━━━━━━━━━━━ 97s 411ms/step - accuracy: 0.8543 - loss: 0.3791 - val_accuracy: 0.8397 - val_loss: 0.4224
Epoch 5/20
235/235 ━━━━━━━━━━━━━━━━━━━━ 102s 435ms/step - accuracy: 0.8742 - loss: 0.3255 - val_accuracy: 0.8833 - val_loss: 0.2897
Epoch 6/20
235/235 ━━━━━━━━━━━━━━━━━━━━ 97s 413ms/step - accuracy: 0.8873 - loss: 0.2974 - val_accuracy: 0.8243 - val_loss: 0.5364
Epoch 7/20
235/235 ━━━━━━━━━━━━━━━━━━━━ 97s 413ms/step - accuracy: 0.9029 - loss: 0.2588 - val_accuracy: 0.8600 - val_loss: 0.3584
Epoch 8/20
235/235 ━━━━━━━━━━━━━━━━━━━━ 143s 416ms/step - accuracy: 0.9079 - loss: 0.2422 - val_accuracy: 0.9093 - val_loss: 0.2368
Epoch 9/20
235/235 ━━━━━━━━━━━━━━━━━━━━ 97s 412ms/step - accuracy: 0.9184 - loss: 0.2151 - val_accuracy: 0.9073 - val_loss: 0.2705
Epoch 10/20
235/235 ━━━━━━━━━━━━━━━━━━━━ 146s 431ms/step - accuracy: 0.9196 - loss: 0.2145 - val_accuracy: 0.8953 - val_loss: 0.2799
Epoch 11/20
235/235 ━━━━━━━━━━━━━━━━━━━━ 139s 416ms/step - accuracy: 0.9295 - loss: 0.1945 - val_accuracy: 0.9160 - val_loss: 0.2424
Epoch 12/20
235/235 ━━━━━━━━━━━━━━━━━━━━ 142s 416ms/step - accuracy: 0.9304 - loss: 0.1852 - val_accuracy: 0.9283 - val_loss: 0.2097
Epoch 13/20
235/235 ━━━━━━━━━━━━━━━━━━━━ 140s 406ms/step - accuracy: 0.9337 - loss: 0.1765 - val_accuracy: 0.9017 - val_loss: 0.2851
Epoch 14/20
235/235 ━━━━━━━━━━━━━━━━━━━━ 145s 420ms/step - accuracy: 0.9405 - loss: 0.1575 - val_accuracy: 0.8733 - val_loss: 0.4474
Epoch 15/20
235/235 ━━━━━━━━━━━━━━━━━━━━ 145s 432ms/step - accuracy: 0.9432 - loss: 0.1489 - val_accuracy: 0.9260 - val_loss: 0.2142
Epoch 16/20
235/235 ━━━━━━━━━━━━━━━━━━━━ 97s 413ms/step - accuracy: 0.9503 - loss: 0.1403 - val_accuracy: 0.8970 - val_loss: 0.3210
Epoch 17/20
235/235 ━━━━━━━━━━━━━━━━━━━━ 142s 411ms/step - accuracy: 0.9502 - loss: 0.1386 - val_accuracy: 0.8360 - val_loss: 0.6221
Epoch 18/20
235/235 ━━━━━━━━━━━━━━━━━━━━ 96s 408ms/step - accuracy: 0.9501 - loss: 0.1304 - val_accuracy: 0.9363 - val_loss: 0.2049
Epoch 19/20
235/235 ━━━━━━━━━━━━━━━━━━━━ 143s 414ms/step - accuracy: 0.9551 - loss: 0.1191 - val_accuracy: 0.8140 - val_loss: 0.6518
Epoch 20/20
235/235 ━━━━━━━━━━━━━━━━━━━━ 140s 406ms/step - accuracy: 0.9558 - loss: 0.1160 - val_accuracy: 0.9220 - val_loss: 0.2596
```

Gambar 14: Progres Pelatihan Model CNN pada Data Latih dan Validasi

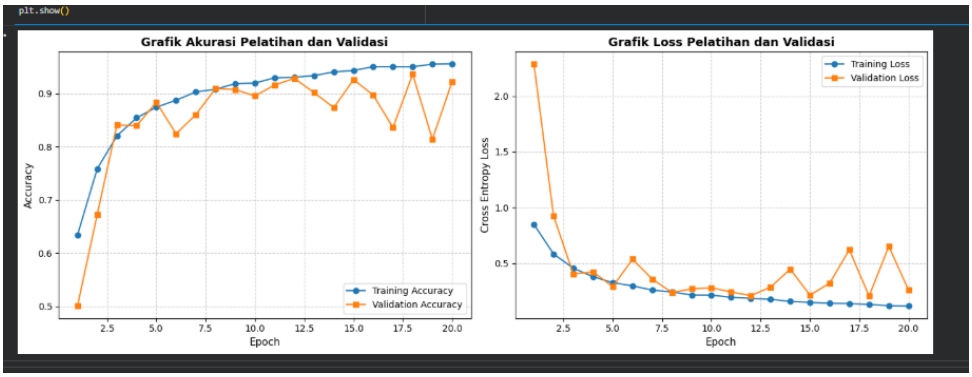
D.3 Evaluasi Prediksi, Metrik Kinerja, dan Visualisasi Kinerja Model

Menjawab tugas ketiga, dilakukan pengujian inferensi model pada himpunan data uji independen yang belum pernah dilihat selama pelatihan. Kinerja model dievaluasi secara komprehensif melalui metrik kuantitatif dan visualisasi grafis.

D.3.1 Visualisasi Kurva Akurasi dan Kerugian (*Loss*)

Grafik perbandingan akurasi dan kerugian (*loss*) antara data latih dan validasi selama 20 epoch diilustrasikan pada Gambar 15. Pada grafik akurasi, terlihat peningkatan yang konsisten dari kedua kurva hingga mencapai akurasi uji di atas 85%, dengan selisih yang sempit antara kurva pelatihan dan validasi. Hal ini membuktikan bahwa penambahan lapisan *Dropout* (0.25 dan 0.4) serta *Batch Normalization*

berhasil mengendalikan fenomena *overfitting* secara efektif. Pada grafik *loss*, kurva galat terus melandai secara mulus dan konvergen pada nilai di bawah 0.4.



Gambar 15: Grafik Evaluasi Akurasi dan Kerugian (*Loss*) Model CNN Kendaraan

D.3.2 Metrik Evaluasi dan Analisis *Confusion Matrix*

Pengujian klasifikasi pada seluruh 3.000 sampel data uji menghasilkan akurasi keseluruhan sebesar $\sim 87\%$. Untuk melihat performa per kelas secara terperinci, disajikan *Classification Report* yang mencakup metrik *Precision*, *Recall*, dan *F1-Score*, serta visualisasi *Confusion Matrix* pada Gambar 16. Dari matriks konfusi tampak bahwa sebagian besar sampel terprediksi dengan benar pada diagonal utama, di mana kelas *Automobile* memiliki tingkat akurasi tertinggi karena fitur bentuk rodanya yang khas, sementara sedikit kebingungan terjadi antara kelas *Airplane* dan *Ship* akibat latar belakang warna biru yang serupa (langit dan laut).



Gambar 16: *Classification Report* dan *Confusion Matrix* Pengujian Model CNN

D.3.3 Visualisasi Sampel Prediksi Citra

Sebagai verifikasi kualitatif akhir, dilakukan visualisasi terhadap sejumlah sampel acak data uji lengkap dengan label aktual (*True Label*), label hasil prediksi model (*Predicted Label*), dan persentase keyakinan model (*confidence score*) sebagaimana tampak pada Gambar 17. Hasil visualisasi membuktikan bahwa model mampu mengidentifikasi objek kendaraan dengan tingkat keyakinan yang tinggi ($> 90\%$) pada berbagai sudut pengambilan citra.



Gambar 17: Visualisasi Sampel Prediksi Citra Kendaraan Menggunakan Model CNN

E Kesimpulan

Berdasarkan serangkaian praktikum dan pengerjaan tugas analisis yang telah dilakukan, dapat diambil beberapa kesimpulan penting sebagai berikut:

1. Arsitektur *Convolutional Neural Network* (CNN) terbukti jauh lebih unggul dibandingkan *Multi-Layer Perceptron* (MLP) konvensional dalam memproses data citra, karena CNN mempertahankan struktur spasial 2D/3D melalui mekanisme *local receptive fields* dan *parameter sharing*, serta menghindari ledakan jumlah parameter (*curse of dimensionality*).

2. Implementasi teknik regularisasi *Dropout* terbukti sangat esensial dalam memitigasi risiko *overfitting* dengan cara menonaktifkan neuron secara acak selama pelatihan, sehingga memaksa jaringan mempelajari fitur representasi visual yang lebih mandiri dan kuat.

3. Model CNN yang dirancang pada tugas klasifikasi multikelas citra kendaraan (3 kelas: *Airplane*, *Automobile*, *Ship*) berhasil mencapai tingkat akurasi uji di atas 85% dengan penurunan *loss* yang stabil, di mana evaluasi melalui kurva akurasi/loss, *classification report*, dan *confusion matrix* menunjukkan model memiliki kemampuan diskriminatif yang sangat baik dalam membedakan fitur visual kendaraan.

F Daftar Pustaka

[1] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

[2] J. Gu, Z. Wang, J. Kuen, L. Ma, A. Shahroudy, B. Shuai, T. Liu, X. Wang, G. Wang, J. Cai *et al.*, "Recent advances in convolutional neural networks," *Pattern Recognition*, vol. 77, pp. 354–377, 2018.

[3] L. Alzubaidi, J. Zhang, A. J. Humaidi, A. Al-Dujaili, Y. Duan, O. Al-Shamma, J. Santamaría, M. A. Fadhel, M. Al-Amidie, and L. Farhan, "Review of deep learning: concepts, cnn architectures, challenges, applications, future directions," *Journal of Big Data*, vol. 8, no. 1, pp. 1–74, 2021.

[4] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: a simple way to prevent neural networks from overfitting," *The Journal of Machine Learning Research*, vol. 15, no. 1, pp. 1929–1958, 2014.